CHAPTER 4: IMPLEMENTATION AND RESULTS

4.1 Overview

This chapter discusses the employed robot platform including the sensors that have been employed and the system design.

4.2 Robot Platform

The developed robot system consists of four main units. ***First***, the sensing unit, which consists of a number of sensors required to sense the robot's surrounding environment. Four sensors are required in the solving robot system which are: ultrasonic sensor, color sensor, and inertial navigation sensor. Each sensor is described below:

1. Ultrasonic sensor: in order for the autonomous robot to navigate the maze area and avoid obstacles, an ultrasonic sensor is used to estimate the distance between the robot and facing wall. The ultrasonic sensor depicted in Figure 4.1 discovers the distance to a solid object by generating an ultrasonic wave that is received back after bouncing off an object. The time it takes to be received back specifies the distance to that object. used to detect and measure distance from objects and send that data to the brick for possible action according to programmed parameters. The ultrasonic can be used in the maze solving system to detect the presence of walls heading the robot.

Figure 4.1: Lego EV3 Ultrasonic sensor

2. Color sensor: this sensor aims to distinguish between the drawn colors on the maze area to determine the position of intersection points. The color sensor presented in Figure 4.2 has the ability of distinguishing between seven various colors (black, white, blue, green, yellow, red, and brown) and sends the data to the brick for possible action according to programmed parameters and also it can

discover the absence of color. Color sensor may benefit the maze solving system through the ability to distinguish between colors drawn on the floor, in order to determine the position of crossways found in the maze area.

Figure 4.2: The Lego Ev3 color sensor

3. Rotation sensor: using this sensor, the robot can obtain its direction in the correct way. The direction of the designed robotic system is essential in order to perform the navigation process in an efficient manner. In our project, we have employed the rotation sensor, which is able to compute the robot's direction when travel between two points, and hence balance the robot direction. Figure 4.3 presents the rotation sensor.

Figure 4.3: The Lego Ev3 rotation sensor (Gyro sensor)

4. Inertial navigation sensor: this sensor is used to estimate the distance travelled by the robot. This sensor helps in finding out the shortest path possible in the maze area. Large motor depicted in Figure 4.4 is able to provide location feedback, since there are encoders attached, which may help in the positioning issue.

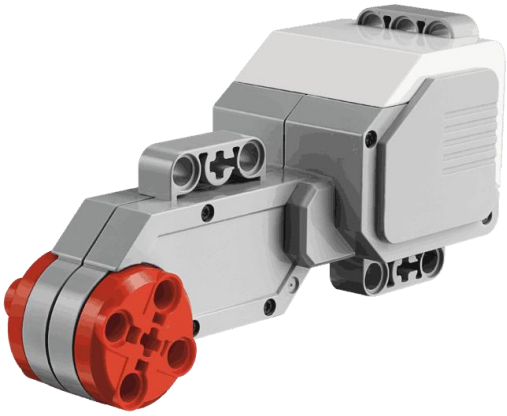

Figure 4.4: Lego EV3 large motor

Second, the computing and control unit. The processing of sensor data and apply the maze solving algorithm are taken place in this unit. The EV3 brick is responsible for collecting the sensed data from sensors, process the sensed data, and then feed them to the maze solving algorithm. The EV3 brick controls the movement of the robot using the EV3 DC motors.

Third, the communication unit. The data collected from sensors (ultrasonic, color, and inertial) are required to be transmitted to a personal computer in order to process the navigation function and to allow online positioning. Data will be transmitted using Bluetooth technology, where EV3 offers reliable Bluetooth communication with nearby devices.

And finally, ***Fourth***, is the power supply unit. Sensing unit, computing and control unit, and communication unit are required to be connected to a power supply in order to accomplish the units' tasks. Power supply unit offers reliable power to all units in the robot system.

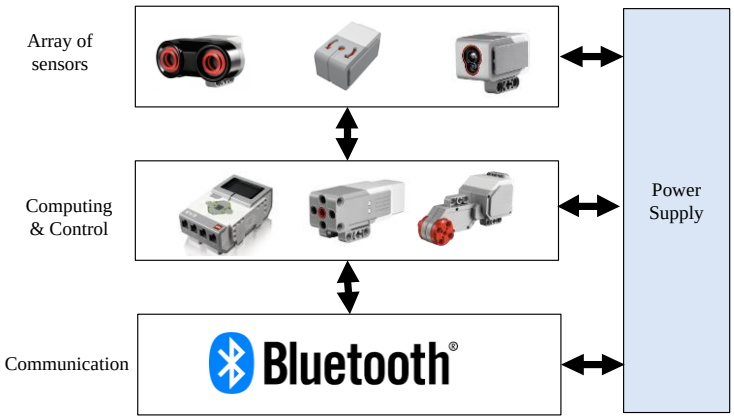

Figure 4.5: Robot architecture

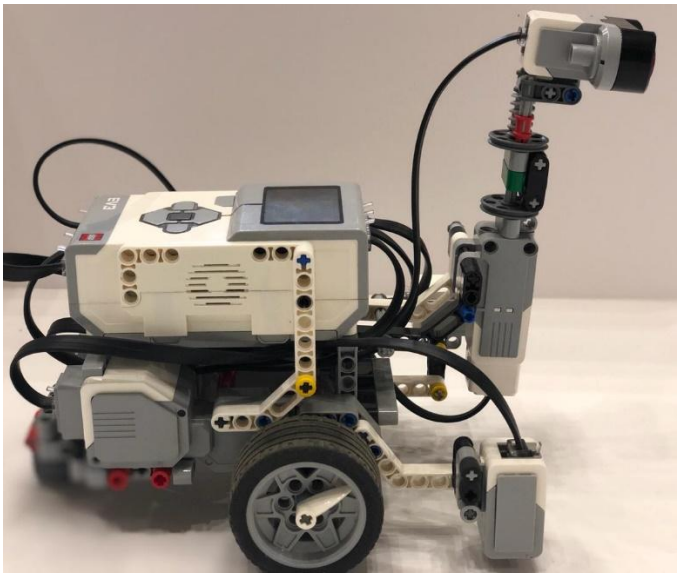

Figure 4.6: The employed robot platform

4.3 Software Requirements

In this, we discuss the software tools that have been employed in this study to accomplish the design and implementation of the robot navigation system. The programming requirements are divided into 3 main components:

1. MicroPython programming language: MicroPython is a streamlined version of the Python programming language designed to run efficiently on microcontrollers and embedded systems. MicroPython provides a powerful and flexible platform for developing hardware-level applications using familiar Python syntax. In the context of LEGO EV3 robotics, MicroPython enables users to program the robot directly on its intelligent brick without relying on external devices or complex setups. Through projects like Pybricks, which builds on MicroPython, developers can control motors, read sensor data, and implement logic for autonomous behaviors on the EV3 platform.
2. Lego API (Pybricks): is a tool to let the developed write and run Python-based programs and power them up on Lego hubs. Pybricks is a Python-based programming environment designed specifically for LEGO robotics platforms. Pybricks allows developers to write code directly in MicroPython, a lightweight version of Python optimized for microcontrollers. Pybricks runs directly on the LEGO hardware without needing a separate operating system, enabling real-

time control of motors, sensors, and other components. It offers a user-friendly API that supports advanced features such as precise motor control, sensor integration, and multitasking.

3. Visual studio code (VS code): is a lightweight, powerful, and highly customizable source-code editor developed by Microsoft. It supports a wide range of programming languages and extensions, including those for embedded development like MicroPython. By connecting Lego EV3 running MicroPython via USB or serial, developers can write, edit, and deploy MicroPython code efficiently within the VS Code environment. This setup offers an integrated workflow ideal for both rapid prototyping and educational use.

4.4 Experiment Testbed

For a proof-of-concept purpose, we have conducted a number of real experiments using two different experiment testbeds, presented in Figures 4.7 and 4.8.

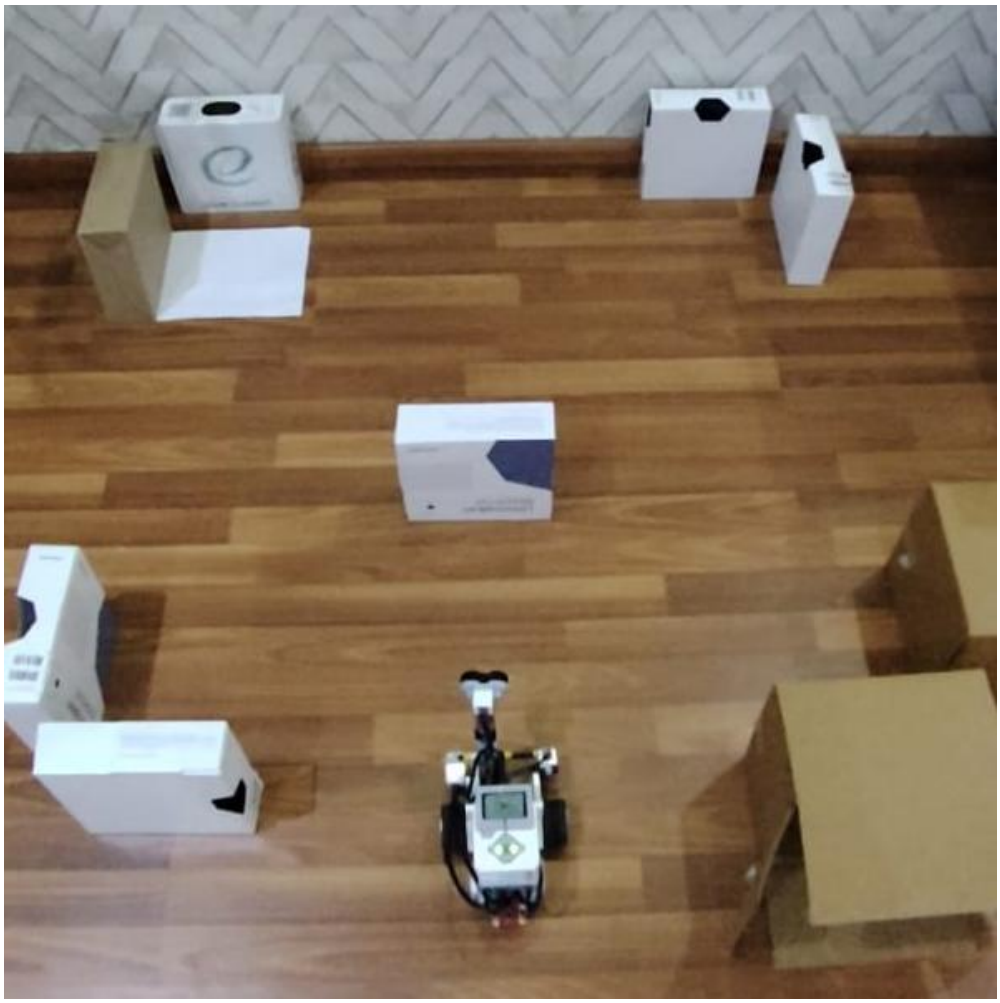

Figure 4.7: Experiment testbed 1

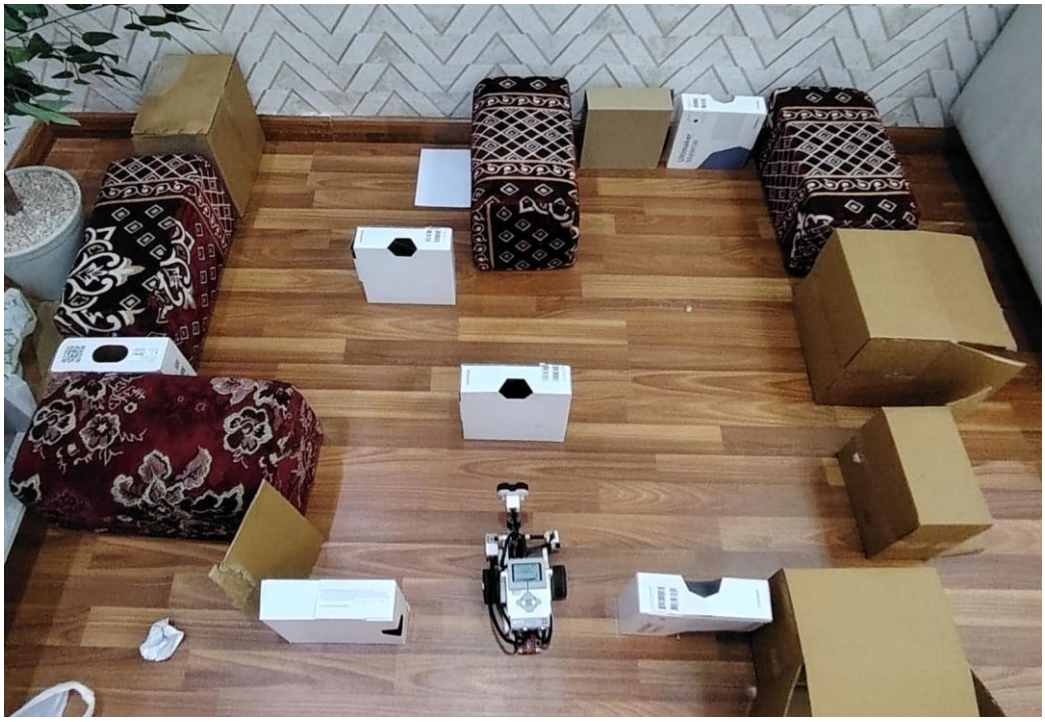

Figure 4.8: Experiment testbed 2

In this project, two different experiment testbeds have been adopted in order validate the efficiency of the presented navigation system. In this project, two different navigation methods have been adopted (right-hand rule and random mouse), where the efficiency for each one was assessed. In addition, A* search algorithm has been implemented and assessed using the above two experiment testbeds.

4.5 Experimental Results

This subsection presents the obtained results from the conducted experimental studies in two different areas. Therefore, we consider the following evaluation metrics:

1. Total travelled distance in the area: this refers to the total distance travelled in the environment.
2. Total travelled distance to the destination-point: this refers to the total distance travelled in the area to the destination-point.
3. Total navigation time: this refers to the total time that is needed to reach navigate the whole experiment testbed.
4. Total time to destination: this refers to the total time needed to reach the destination point.

First, the total travelled distance evaluation metric is considered. Table 4.1 presents the distance travelled in the navigation area for each navigation method in the two

experiment testbeds. Both the right-hand rule and the random mouse methods travelled 230 cm in testbed 1, however, the right-hand rule failed in testbed 2 since the robot entered an infinity loop, and hence fail the navigation method. On the other hand, the random mouse method travelled 439 cm in testbed 2.

Table 4.1: Experimental results for the total travelled distance (cm) in each testbed

Navigation method	**Testbed 1**	**Testbed 2**
Right-hand-rule	230	Infinity loop
Random mouse	230	439

Second, the total travelled distance to the target point in each experiment is discussed. Table 4.2 shows the total distance travelled in centimeter (cm) to the target point. For instance, both the right-hand rule, and the random mouse methods travelled in 131 cm to reach the destination point. However, the right-hand rule failed in the experiment testbed 2 as it entered in an infinity loop, whereas the random mouse method travelled in 210 cm to successfully reach the target point.

Table 4.2: Experimental results for the total travelled distance (cm) to the target point in each testbed

Navigation method	**Testbed 1**	**Testbed 2**
Right-hand-rule	131 cm	Infinity loop
Random mouse	131 cm	210 cm

Third, the total navigation time is assessed for the implemented two navigation methods, in two different experiment testbeds. For instance, the time required to navigate the experiment testbed 1 using the right-hand-rule was around 140 seconds, whereas 119 seconds were needed to navigate the experiment testbed 1 using the random mouse method. This refers to the method behaviour, as the right-hand-rule works through navigating the right directions first, and then search in the left direction. However, the random mouse method works through moving in a random direction.

In the experiment testbed 2, the random mouse algorithm requires 341 seconds to navigate the whole environment, whereas the right-hand rule was not successful in exploring the environment due to the occurrence of infinity loop issue. The obtained results are presented in Table 4.3.

Table 4.3: Experimental results for the total time required to navigate the whole environment

Navigation method	**Testbed 1**	**Testbed 2**
Right-hand-rule	140	Infinity loop
Random mouse	119	341

On the other hand, the total time required to allow the robot to reach the destination point was estimated in each experiment testbed, as presented in Table 4.4. The right-hand rule required 78 seconds to reach the target point in the first experiment testbed, where it failed in the second experiment testbed, as discussed earlier. However, the random mouse algorithm was faster with 42 seconds to reach the target point for the first experiment testbed, and 104 seconds to reach the target point in the second experiment testbed.

Table 4.4: Experimental results for the total time required to reach the destination

Navigation method	**Testbed 1**	**Testbed 2**
Right-hand-rule	78	Infinity loop
Random mouse	42	104

Finally, the effectiveness of the A* search algorithm is presented in Table 4.5, where the robot required 131 seconds to reach the target points using the A* searching algorithm, whereas the robot requires 210 seconds to reach the target point in the experiment testbed 2.

Table 4.5: Experimental results for the total time required to reach the destination using A* search algorithm

Search algorithm	**Testbed 1**	**Testbed 2**
A*	131	210

REFERENCES

[1] Experimentation on the motion of an obstacle avoiding robot

[2] Design and Implementation of a Real-Time Autonomous Navigation System Applied to Lego Robots

[3] https://en.m.wikipedia.org/wiki/Dijkstra%27s_algorithm

[4] https://en.wikipedia.org/wiki/Maze-solving_algorithm

[5] https://ocw.mit.edu/courses/6-034-artificial-intelligence-fall-2010/resources/lecture-4-search-depth-first-hill-climbing-beam/

[6] Design and Implementation of a Real-Time Autonomous Navigation System Applied to Lego Robots

[7] Autonomous Navigation of a Lego Robot with Neural Networks

[8] A Comparative Study of Bug Algorithms for Robot Navigation

[9] Autonomous Navigation of Mobile Robot Based on Flood Fill Algorithm

[10] Experimentation on the motion of an obstacle avoiding robot

[11] Maze solving robot with automated obstacle avoidance

[12] AnAutonomousMaze-Solving Robotic System Based on an Enhanced Wall-Follower Approach

[13] Shortest path traversal in a maze with wall following robot

[14] https://github.com/resources/articles/software-development/what-is-sdlc

[15] https://artoftesting.com/waterfall-model

[16] https://deeprojectmanager.com/functional-vs-non-functional-requirements/